Gödel's First and Second Incompleteness Theorems in a Guard/Rose Equational System: Formalization in Agda

Abstract

We formalize Gödel's first and second incompleteness theorems inside a purely equational system combining Guard's tree/functor syntax with Rose's equational proof style. The ambient meta-theory is Agda with `--safe --without-K --exact-split`. No postulates.

The system has no logical connectives: the only judgement is an equation $t = u$ of terms; rules are computation axioms, congruence, substitution, a tree-induction schema, and a single hypothesis rule. We add one new primitive (RecP) in order to give the object-level proof evaluator a genuine hypothesis-awareness, closing two loopholes that made an earlier formalization (V2) prove a strictly weaker theorem.

The main constructive statement is:

$$\text{conSentenceV3} \vdash \mathbf{t} = \mathbf{f},$$

the internal consistency sentence, taken as the only axiom, proves bottom. Equivalently, if the Gödel sentence gs proves its own internal consistency, then gs is inconsistent.

Contents

1 Overview **1**

2 The formal system **2**
- 2.1 Trees . . . 2
- 2.2 Terms and functors . . . 2
- 2.3 Embedding trees into terms . . . 2
- 2.4 Derivation rules . . . 2
- 2.5 Meta-level consistency . . . 4

3 Encoding terms, equations, and derivations **4**
- 3.1 Term and equation codes . . . 4
- 3.2 Encoded derivations . . . 4

4 The object-level proof evaluator thmT **4**
- 4.1 What thmT$(h)(x)$ computes . . . 5
- 4.2 Soundness of the encoding (Theorem 14, V3) . . . 5

5 Consistency: meta and object **5**

6 The Gödel sentence and the diagonal construction **6**
6.1 Informal picture . . . 6
6.2 The template . . . 6
6.3 The self-code and the replacement tree . . . 6
6.4 The V3 Gödel sentence . . . 7
6.5 Why keep x_2, x_{11}, x_{12} free . . . 7
6.6 Diagonal identity . . . 7

7 Gödel's First Incompleteness Theorem **8**

8 Gödel's Second Incompleteness Theorem **8**
8.1 Auxiliary: the encoded $\perp$-proof . . . 8
8.2 Positive form . . . 9
8.3 Contrapositive . . . 9
8.4 Sharpest form . . . 9

9 Negative acceptance test **9**

A Design rationale for RecP **9**

B Connections with predecessors **10**
B.1 Guard (1963) [1] . . . 10
B.2 Rose (1961) [2] . . . 10
B.3 Hilbert–Bernays derivability conditions . . . 10
B.4 Joyal's approach [3] . . . 10
B.5 V2 of this formalization . . . 11

C File index **11**

1 Overview

The formalization uses two deliberate design choices:

- *Guard's* binary trees with unary and binary functors, as both data and term language [1].
- *Rose's* purely equational proof style [2]: no connectives, no sequents, no logical rules — only computation, congruence, substitution, Schema F, and a rule for a single equational hypothesis.

This is strictly weaker than either author's setting (Guard's BRA has $\sim, \supset$, modus ponens and a Hilbert-style induction; Rose works with unary numerals), and the classical Hilbert–Bernays route to Gödel II is unavailable: there is no implication connective and no modus ponens. We show that the diagonal argument adapts, via *polymorphic reflection* and a strong coding of derivations, to yield a sharp positive statement of Gödel II.

The version V3 of the formalization (described here) replaces an earlier V2 evaluator that had two encoding loopholes; the old V2 development is preserved in `Guard/archive/` for comparison.

2 The formal system

2.1 Trees

The ground data are finite binary trees:

$$a, b \ ::= \ O \ \mid \ \langle a, b \rangle$$

with O the leaf and $\langle a, b \rangle$ the node. We stipulate:

$$\mathbf{t} = O, \qquad \mathbf{f} = \langle O, O \rangle.$$

These are structurally distinct trees; "inconsistent" at the meta level means the equation $\mathbf{t} = \mathbf{f}$ is derivable. Natural numbers are encoded unarily: $\overline{0} = O$, $\overline{n+1} = \langle O, \overline{n} \rangle$.

2.2 Terms and functors

Terms are built from trees, variables, and functor applications. We write t, u, v for terms, a, b for trees. The system has countably many variables $x_0, x_1, x_2, \ldots$.

Unary functors ($f, g : \mathrm{Fun}_1$) and **binary functors** ($h, s : \mathrm{Fun}_2$):

f, g	$::=$	I	identity
	$\mid$	$\pi_1,\ \pi_2$	projections
	$\mid$	$f \circ g$	composition
	$\mid$	$h[f, g]$	binary composition
	$\mid$	$\mathrm{R}(z, s)$	tree recursion
	$\mid$	K_c	constant
h, s	$::=$	P	pair
	$\mid$	C	left projection
	$\mid$	$\mathrm{Lf}(f)$	lift: $(t, u) \mapsto f(t)$
	$\mid$	$\mathrm{Ps}(f, h)$	post: $(t, u) \mapsto f(h(t, u))$
	$\mid$	$\mathrm{Fn}(h_1, h_2, h)$	fan: $(t, u) \mapsto h(h_1(t, u), h_2(t, u))$
	$\mid$	IfLf	conditional
	$\mid$	TreeEq	tree equality
	$\mid$	$\mathrm{RecP}(s)$	parameterised tree recursion (V3 addition)

Terms:

$$t, u \ ::= \ O \ \mid \ x_i \ \mid \ f(t) \ \mid \ h(t, u).$$

An *equation* is a pair $t = u$.

2.3 Embedding trees into terms

Every tree a determines a closed term $\widehat{a}$ by $\widehat{O} = O$, $\widehat{\langle a, b \rangle} = \langle \widehat{a}, \widehat{b} \rangle$. (In Agda: `reify`.)

2.4 Derivation rules

Derivations use a single equational hypothesis H (another equation); we write $H \vdash t = u$.

Computation axioms.

$$
\begin{array}{lr}
\text{Identity:} & \mathrm{I}(t) = t \\
\text{Projections:} & \pi_1(\langle t, u\rangle) = t, \qquad \pi_2(\langle t, u\rangle) = u \\
\text{Constant:} & \mathrm{C}(t, u) = t \\
\text{Composition:} & (f \circ g)(t) = f(g(t)) \\
\text{Bin. composition:} & h[f, g](t) = h(f(t),\, g(t)) \\
\text{Lift:} & \mathrm{Lf}(f)(t, u) = f(t) \\
\text{Post:} & \mathrm{Ps}(f, h)(t, u) = f(h(t, u)) \\
\text{Fan:} & \mathrm{Fn}(h_1, h_2, h)(t, u) = h(h_1(t, u),\, h_2(t, u)) \\
\text{K constant:} & \mathrm{K}_c(t) = c
\end{array}
$$

Tree recursion.

$$
\begin{array}{ll}
\text{Base:} & \mathrm{R}(z, s)(O) = z \\
\text{Step:} & \mathrm{R}(z, s)(\langle t, u\rangle) = s(\langle t, u\rangle,\ \langle \mathrm{R}(z, s)(t), \mathrm{R}(z, s)(u)\rangle)
\end{array}
$$

Parameterised tree recursion (V3).

$$
\begin{aligned}
&\mathrm{RecP}(s)(p, O) &&= O \\
&\mathrm{RecP}(s)(p, \langle a, b\rangle) = &&s(\langle p, \langle a, b\rangle\rangle,\ \langle \mathrm{RecP}(s)(p, a), \mathrm{RecP}(s)(p, b)\rangle)
\end{aligned}
$$

$\mathrm{RecP}(s)$ is a binary functor. It recurses on its second argument while carrying the first argument p unchanged, making p available inside the step function at every node. See §A for the design rationale.

Conditional.

$$
\begin{aligned}
\mathrm{IfLf}(O,\ \langle t, u\rangle) &= t \\
\mathrm{IfLf}(\langle a, b\rangle,\ \langle t, u\rangle) &= u
\end{aligned}
$$

Tree equality.

$$
\begin{aligned}
&\mathrm{TreeEq}(O, O) &&= \mathbf{t} \\
&\mathrm{TreeEq}(O, \langle t, u\rangle) &&= \mathbf{f} \\
&\mathrm{TreeEq}(\langle t, u\rangle, O) &&= \mathbf{f} \\
&\mathrm{TreeEq}(\langle t_1, t_2\rangle, \langle u_1, u_2\rangle) = &&\mathrm{IfLf}(\mathrm{TreeEq}(t_1, u_1),\ \langle \mathrm{TreeEq}(t_2, u_2), \mathbf{f}\rangle)
\end{aligned}
$$

Structural rules.

$$
\frac{}{t = t}\,\mathrm{refl} \qquad \frac{t = u}{u = t}\,\mathrm{sym} \qquad \frac{t = u \qquad u = v}{t = v}\,\mathrm{trans}
$$

$$
\frac{t = u}{f(t) = f(u)}\,\mathrm{cong}_1 \qquad \frac{t = u}{h(t, v) = h(u, v)}\,\mathrm{cong}_L \qquad \frac{t = u}{h(v, t) = h(v, u)}\,\mathrm{cong}_R
$$

Substitution.

$$
\frac{H \vdash t = u}{H \vdash t[c/x_i] = u[c/x_i]}\,\mathrm{inst}
$$

Hypothesis.

$$
\frac{}{H \vdash H}\,\mathrm{hyp}
$$

Schema F (tree induction). If $f, g : \mathrm{Fun}_1$, z a term, $s : \mathrm{Fun}_2$, and

$$f(O) = z, \qquad f(\langle x_0, x_1\rangle) = s(\langle x_0, x_1\rangle,\ \langle f(x_0), f(x_1)\rangle),$$
$$g(O) = z, \qquad g(\langle x_0, x_1\rangle) = s(\langle x_0, x_1\rangle,\ \langle g(x_0), g(x_1)\rangle),$$

then $f(x_0) = g(x_0)$. Schema F is *uniqueness of tree recursion*: two functions satisfying the same recurrence agree everywhere.

2.5 Meta-level consistency

Definition 1 (Meta-consistency)**.** *Consistent*(H) *means there is no derivation of* $\mathbf{t} = \mathbf{f}$ *from* H*. Formally (Agda):*

$$\mathit{Consistent}(H) \ \coloneqq\ (H \vdash \mathbf{t} = \mathbf{f}) \to \bot.$$

Note: *Consistent* is an Agda-level predicate (a `Set`), *not* a formal statement of the object theory. The object-level counterpart is conSentenceV3 (Definition 3).

3 Encoding terms, equations, and derivations

Every syntactic object has a *tree code*.

3.1 Term and equation codes

Mutually recursive codings code : Term $\to$ Tree, codeF_1, codeF_2 on functors, with 29 tag values assigning distinct trees to each term constructor and each functor:

$$\begin{aligned}
\mathrm{code}(O) &= \langle \mathrm{tagO}, O\rangle \\
\mathrm{code}(x_n) &= \langle \mathrm{tagVar}, \overline{n}\rangle \\
\mathrm{code}(f(t)) &= \langle \mathrm{tagAp}_1, \langle \mathrm{codeF}_1(f), \mathrm{code}(t)\rangle\rangle \\
\mathrm{code}(h(t,u)) &= \langle \mathrm{tagAp}_2, \langle \mathrm{codeF}_2(h), \langle \mathrm{code}(t), \mathrm{code}(u)\rangle\rangle\rangle
\end{aligned}$$

For equations: $\mathrm{codeEqn}(t = u) = \langle \mathrm{code}(t), \mathrm{code}(u)\rangle$.

We write $\ulcorner t \urcorner$ for $\mathrm{code}(t)$ when unambiguous; $\ulcorner H \urcorner$ for $\mathrm{codeEqn}(H)$. The *reified code* of a term t is the closed term $\widehat{\ulcorner t \urcorner}$; similarly for equations.

3.2 Encoded derivations

Every derivation $d : H \vdash t = u$ has a tree-encoding $\mathrm{enc}(d)$, computed by a function thm14EV3 defined in `Guard/Thm14EV3.agda` that dispatches on the constructor of d. Each of the 29 derivation-constructors has a distinct tag $n_0 \ldots n_{28}$; the encoded tree has this tag as its outer left child and the relevant sub-data (including encoded sub-derivations) as its right child.

For hyp: $\mathrm{enc}(\mathrm{hyp}_H) = \langle \overline{26}, \widehat{\ulcorner H \urcorner}\rangle$.

4 The object-level proof evaluator thmT

thmT : Term $\to$ Fun_1 is a meta-level function producing a specific closed unary functor for each Term argument. Its defining equation:

$$\mathrm{thmT}(h) \ =\ \mathrm{R}(O,\ \mathrm{thmTStep}(h))$$

where $\mathrm{thmTStep}(h) : \mathrm{Fun}_2$ is a 29-branch Fan-dispatch on the outer tag of its first argument. Every branch is closed except the *hypothesis branch*, which has h embedded as a K_h-like constant.

4.1 What $\mathrm{thmT}(h)(x)$ computes

Informally, $\mathrm{thmT}(h)(x)$ is a *decoder*: given a Term h interpreted as the reified code of a hypothesis H and a Term x interpreted as a proof-tree encoding, it returns the reified codeEqn of the decoded conclusion, or the sentinel O if the input shape is ill-formed.

- $x = O$: returns O.
- $x = \langle \overline{n}, \mathrm{body} \rangle$ for $n \in \{0, \ldots, 28\}$: dispatches to the n-th case. Each case_n extracts sub-data from body and produces the reified codeEqn corresponding to rule n.
- $n = 26$ (the *hypothesis case*): checks body $\equiv h$ via TreeEq; on match returns h, on mismatch returns O.
- Any other shape: returns O.

$\mathrm{thmT}(h)(x)$ is not a boolean test: it returns equation codes, with the sentinel O reserved for invalid inputs. Reified equation codes are always pair-shaped, hence disjoint from O.

4.2 Soundness of the encoding (Theorem 14, V3)

Theorem 2 (`thm14EV3` in `Guard/Thm14EV3.agda`)**.** *For every derivation $d : H \vdash t = u$, the encoding* $\mathrm{enc}(d) : \mathrm{Term}$ *satisfies, in any ambient hypothesis context:*

$$\vdash \ \mathrm{thmT}(\widehat{\ulcorner H \urcorner})\,(\mathrm{enc}(d)) = \widehat{\ulcorner t = u \urcorner}.$$

The crucial adjective is *polymorphic in the ambient hypothesis*: the derivation on the right-hand side uses only computation axioms, congruences, substitution, and Schema F — never the hypothesis rule. This is what makes the Gödel II argument work: the encoding's correctness is an *absolute* equation, not conditional on any outer assumption.

The converse (decoder-faithfulness: every x such that $\mathrm{thmT}(\widehat{\ulcorner H \urcorner})(x) = \widehat{\ulcorner t = u \urcorner}$ comes from a genuine derivation) is *not* proved and is not needed. The V3 shape-checks in case26 (hypothesis match) and case19V3 (middle-term match in trans) are sufficient to close the specific loopholes that made the V2 consistency sentence refutable by pure computation.

5 Consistency: meta and object

Definition 3 (Internal consistency statement)**.** *For a specific hypothesis H and its Gödel sentence code $\widehat{\ulcorner H \urcorner}$, the* internal consistency statement *is the single Equation*

$$\mathrm{Con}_H \ \coloneqq \ \big(\mathrm{TreeEq}(\mathrm{thmT}(\widehat{\ulcorner H \urcorner})(x_0),\ \widehat{\ulcorner \mathbf{t} = \mathbf{f} \urcorner}) = \mathbf{f}\big)$$

with one free variable x_0 (the "proof slot"). In the V3 formalization with $H = \mathrm{gs}_{V3}$ the Gödel sentence, the Equation is called conSentenceV3*.*

Since inst allows any Term to be substituted for x_0, asserting Con_H is equivalent to asserting the universal *"for every Term x,* $\mathrm{thmT}(\widehat{\ulcorner H \urcorner})(x) \neq \widehat{\ulcorner \mathbf{t} = \mathbf{f} \urcorner}$*"*. Unpacking: *"no encoded proof, decoded under the H-evaluator, produces $\widehat{\ulcorner \mathbf{t} = \mathbf{f} \urcorner}$ as its conclusion"*. That is Con_H reflected inside the object language.

Two layers of consistency coexist:

- **Meta-level**: *Consistent*(H) is an Agda predicate asserting the empty type of derivations of $\mathbf{t} = \mathbf{f}$ from H.
- **Object-level**: Con_H is a formal Equation, expressible precisely because the equational system can represent its own decoder via thmT.

6 The Gödel sentence and the diagonal construction

This is a standard Carnap-style diagonalisation, rendered at the tree-code level. The goal is a self-referential sentence — an equation that, informally, "talks about its own code".

6.1 Informal picture

Classically (Carnap 1934, Gödel 1931): given a one-free-variable formula $\phi(x)$, the diagonal lemma constructs a sentence ψ such that $\psi \iff \phi(\ulcorner\psi\urcorner)$. The key move is substituting a numeral for the formula's own code into a specifically-crafted template.

We adapt this to the equational setting as follows. A *template equation* with a free "self-code slot" x_1 is written down. A specific substitution instance of this template, with x_1 replaced by a concrete Term representing the template's own code, yields the Gödel sentence. A *diagonal identity* then says: applying substOp (the object-level substitution operator) to the template's code reproduces the Gödel sentence's code. This is the fixed-point equation that makes the sentence self-referential.

6.2 The template

The template uses five free variables, each with a distinct role:

x_0	proof slot — will receive a proof-encoding.
x_1	self-code slot — will be replaced by the template's own code (as a Term).
x_2	evaluator hCode — will be bound to $\widehat{\ulcorner \mathrm{gs}_{V3} \urcorner}$ via ruleInst.
x_{11}	substOp replacement parameter — will be bound to $\widehat{\mathrm{crTC}}$.
x_{12}	substOp variable-index parameter — will be bound to $\widehat{\bar{1}}$.

$$\text{templateEqn} \;:=\; \Big(\mathrm{TreeEq}\big(\mathrm{thmT}(x_2)(x_0),\;\; \mathrm{substOp}(\langle x_{11}, x_{12}\rangle,\; x_1)\big) \;=\; \mathbf{f}\Big).$$

The two layers of parameters (x_{11}, x_{12} and x_2) are kept free *on purpose*. Baking them into the template would force the template's own code to contain the specific substituted values, creating a circular definition (see §6.5 below). Keeping them free means the template's tree-code stays small and self-contained; their values are bound later, by ruleInst at proof-construction time.

6.3 The self-code and the replacement tree

Two closely-related trees enter the diagonal; distinguishing them is the key conceptual step.

Definition 4 (Template code)**.**

$$N \;:=\; \ulcorner \text{templateEqn} \urcorner \;\in\; \mathrm{Tree}.$$

This is the tree obtained by applying codeEqn *to the template Equation. It is the "Gödel number" of the template (in tree form).*

Definition 5 (Replacement tree)**.**

$$\mathrm{crTC} \coloneqq \ulcorner \widehat{N} \urcorner \in \mathrm{Tree}.$$

This is the tree obtained by reifying N *into a closed Term* $\widehat{N}$*, and then coding* that Term *back to a Tree.*

Why are N and crTC different? Because the coding function code : Term $\to$ Tree and the reification $\widehat{\cdot}$: Tree $\to$ Term are *not* inverses. Reification just wraps a tree as a nested pair-Term (every nd becomes an ap_2Pair), whereas code attaches constructor-tag information (each ap_2 becomes $\mathrm{nd}\,\mathrm{tagAp}_2\,(\mathrm{nd}\ldots)$). Concretely:

$$\mathrm{code}(\widehat{\mathrm{lf}}) \;=\; \mathrm{code}(O) \;=\; \mathrm{nd}\,\mathrm{lf}\,\mathrm{lf} \;\neq\; \mathrm{lf},$$

so code-$\circ$-reify strictly inflates trees. In particular crTC is larger than N.

What crTC is for. The V3 Gödel sentence is obtained by substituting $\widehat{N}$ for x_1 at the *Term* level:

$$\mathrm{gs}_{V3} \coloneqq \mathrm{templateEqn}[\widehat{N}/x_1].$$

At the *tree-code* level, this corresponds to replacing each occurrence of $\ulcorner x_1 \urcorner = \mathrm{nd}\,\mathrm{tagVar}\,\overline{1}$ inside N by the tree-code of the term being plugged in. That tree-code is precisely $\mathrm{code}(\widehat{N}) = \mathrm{crTC}$.

So N is the target of the coded substitution, and crTC is the replacement. This is the standard *diagonal lemma* pattern: a tree-code-level substitution that plugs the code of the template-plus-its-own-code back into the template.

6.4 The V3 Gödel sentence

$$\mathrm{gs}_{V3} \coloneqq \mathrm{templateEqn}[\widehat{N}/x_1].$$

Only x_1 is substituted; x_0, x_2, x_{11}, x_{12} remain free.

6.5 Why keep x_2, x_{11}, x_{12} free

If we baked x_{11} and x_{12} into the template before computing $N = \ulcorner \mathrm{templateEqn} \urcorner$, then N would contain the specific Term values $\widehat{\mathrm{crTC}}$ and $\widehat{\overline{1}}$ literally. But crTC is defined in terms of N: it is $\ulcorner \widehat{N} \urcorner$. So N would depend on itself, producing a circular definition with no fixed-point.

Keeping x_{11}, x_{12} free breaks the circularity: N references these as abstract free variables; crTC is then computed from the fixed N; the parameters are supplied via ruleInst only at the point where the diagonal identity is used inside the proof.

Similarly, x_2 stays free because V3's evaluator $\mathrm{thmT}(h)$ is hypothesis-specific (via case26), so the evaluator's hCode must match whatever the corr component of an encoded proof produces. V2 did not need an x_2 slot because its evaluator thFunTFor was hypothesis-agnostic.

6.6 Diagonal identity

Lemma 6 (`diagFTargetV3` in `Guard/GodelIV3.agda`)**.** *Let* $\mathrm{cGS} = \ulcorner \mathrm{gs}_{V3} \urcorner$*. Then:*

$$\vdash\ \mathrm{substOp}\big(\langle \widehat{\mathrm{crTC}}, \widehat{\overline{1}} \rangle,\ \widehat{N}\big) \;=\; \widehat{\mathrm{cGS}}.$$

Read: at the object level, applying substOp — with parameters $(\widehat{\mathrm{crTC}}, \widehat{\overline{1}})$ — to the reified template code $\widehat{N}$ yields the reified Gödel-sentence code $\widehat{\mathrm{cGS}}$. This is the fixed-point identity making the construction self-referential.

Proof sketch. At the tree level, $\mathrm{codedSubst}(\mathrm{crTC}, \overline{1}, N) = \mathrm{cGS}$, because cGS *is* the result of substituting $\widehat{N}$ for x_1 in templateEqn, and this substitution at the Term level pushes through to codedSubst at the code level (this is the content of the V2 correctness lemmas, re-used unchanged). Lifting to the object level: substOpEquiv bridges substOp to the older closed substitution cstf, and closedSTFNd evaluates $\mathrm{cstf}(\widehat{N})$ to $\widehat{\mathrm{cGS}}$. Full Agda proof: 42 lines. □

7 Gödel's First Incompleteness Theorem

Theorem 7 (Gödel I, constructive form — `godelIDerivV3`)**.**

$$\mathrm{gs}_{V3} \vdash \mathbf{t} = \mathbf{f}.$$

Proof sketch. Work inside the ambient gs_{V3}. Let $d_G = \mathrm{hyp}_{\mathrm{gs}_{V3}}$. Apply Theorem 2 to d_G, obtaining a specific encoding e and a witness corr polymorphic in ambient hypothesis:

$$\vdash \mathrm{thmT}(\widehat{\ulcorner \mathrm{gs}_{V3} \urcorner})(e) = \widehat{\ulcorner \mathrm{gs}_{V3} \urcorner}.$$

Now instantiate gs_{V3} via ruleInst:

1. $x_2 \mapsto \widehat{\ulcorner \mathrm{gs}_{V3} \urcorner}$ — align the evaluator's hCode with corr's witness.
2. $x_{11} \mapsto \widehat{\mathrm{crTC}}$.
3. $x_{12} \mapsto \widehat{\overline{1}}$.
4. $x_0 \mapsto e$ — plug the encoded proof into the proof slot.

The conclusion becomes

$$\mathrm{gs}_{V3} \vdash \mathrm{TreeEq}\big(\mathrm{thmT}(\widehat{\ulcorner \mathrm{gs}_{V3} \urcorner})(e),\ \mathrm{substOp}(\langle \widehat{\mathrm{crTC}}, \widehat{\overline{1}} \rangle,\ \widehat{N})\big) = \mathbf{f}.$$

By corr the first TreeEq argument reduces to $\widehat{\ulcorner \mathrm{gs}_{V3} \urcorner}$; by Lemma 6 the second does too. Schema F gives $\mathrm{TreeEq}(\widehat{\ulcorner \mathrm{gs}_{V3} \urcorner}, \widehat{\ulcorner \mathrm{gs}_{V3} \urcorner}) = \mathbf{t}$ (via `treeEqSelf`), so the derivation concludes $\mathbf{t} = \mathbf{f}$. □

Corollary 8 (Gödel I, standard form)**.** *If Consistent(H) and $H \vdash \mathrm{gs}_{V3}$, then $\bot$.*

8 Gödel's Second Incompleteness Theorem

Three progressively-sharper formulations of Gödel II, all proved from the same underlying argument. Let $\mathrm{gs} = \mathrm{gs}_{V3}$ and $\mathrm{Con} = \mathrm{conSentenceV3}$.

8.1 Auxiliary: the encoded ⊥-proof

Lemma 9 (`provBot` in `Guard/GodelIIV3.agda`)**.** *There is a specific closed Term $e_\bot$ such that, in any ambient hypothesis:*

$$\vdash \mathrm{thmT}(\widehat{\ulcorner \mathrm{gs} \urcorner})(e_\bot) \ = \ \widehat{\ulcorner \mathbf{t} = \mathbf{f} \urcorner}.$$

This is the necessitation of Theorem 7. The equation on the right is *hypothesis-polymorphic*: it is derivable without any assumption beyond the base equational system.

8.2 Positive form

Theorem 10 (`godelIIPositive`)**.**

$$\mathrm{gs} \vdash \mathrm{Con} \implies \mathrm{gs} \vdash \mathbf{t} = \mathbf{f}.$$

Proof. From $\mathrm{gs} \vdash \mathrm{Con}$ instantiate x_0 with $e_\perp$:

$$\mathrm{gs} \vdash \ \mathrm{TreeEq}(\mathrm{thmT}(\widehat{\ulcorner \mathrm{gs} \urcorner})(e_\perp),\ \widehat{\ulcorner \mathbf{t} = \mathbf{f} \urcorner}) = \mathbf{f}.$$

Rewrite the inner thmT-application using Lemma 9 (cong_L): both TreeEq arguments become $\widehat{\ulcorner \mathbf{t} = \mathbf{f} \urcorner}$. Schema F gives $\mathrm{TreeEq}(c, c) = \mathbf{t}$, so $\mathrm{gs} \vdash \mathbf{t} = \mathbf{f}$. □

8.3 Contrapositive

Theorem 11 (`godelIIV3`)**.** *Consistent*(gs) $\implies \mathrm{gs} \nvdash \mathrm{Con}$.

Immediate from Theorem 10.

8.4 Sharpest form

Theorem 12 (`godelIIConAlone`)**.**

$$\mathrm{Con} \ \vdash \ \mathbf{t} = \mathbf{f}.$$

The internal consistency sentence Con*, taken as the only axiom, proves* $\perp$*.*

Proof. Apply Theorem 10 with ambient hypothesis Con itself; the premise $\mathrm{Con} \vdash \mathrm{Con}$ is the hypothesis rule. □

Remark. The sharpest form is not available classically over PA. It uses the fact that our gs_{V3} is deliberately inconsistent by diagonal construction (Theorem 7), so the encoded $\perp$-proof $e_\perp$ actually exists and is verifiable. PA, conjecturally consistent, admits no such concrete witness — hence classical Gödel II is stated contrapositively. In our setting the positive witness is available and the sharpest statement is just Theorem 12.

9 Negative acceptance test

`Guard/GodelIITestV3.agda` documents that the V2 attack of building a ProofE3 $H\,(\mathbf{t} = \mathbf{f})$ from $\mathrm{trans}(\mathrm{refl}\,\mathbf{t}, \mathrm{refl}\,\mathbf{f})$ is rejected by Agda's type-checker. The two sub-proofs have middle terms $\mathbf{t} = O$ and $\mathbf{f} = \langle O, O \rangle$, which do not unify; thm14EV3Trans's type-signature requires them to agree. This would have succeeded in V2 but does not in V3.

A Design rationale for RecP

RecP is not derivable from the V2 combinators as a Fun_2 expression. Informally:

- $\mathrm{R}(z, s)$ takes a single recursion target; its step s receives $(\mathrm{currentNode}, \mathrm{recs})$ — no sideband parameter.
- Packing a parameter p with the target t into a pair $\langle p, t \rangle$ and applying R causes R to recurse into *both* children, including p's substructure — wrong semantics.

- Embedding p via x_0-placeholder and ruleInst at the Deriv level does work, but produces a *family of* Fun_1 indexed by p, not a single Fun_2 that handles runtime (p, t).

RecP gives the combinator algebra a genuine parameterized-recursion primitive. It does not extend the class of *computable* functions (the class is already primitive recursion with parameters), only the class of closed Fun_2 *expressions*. This matters because the V3 evaluator's proof-dispatch case n_{23} (the ruleInst case) requires a closed Fun_2 applying substitution dynamically to a runtime pair $\langle \widehat{\ulcorner t \urcorner}, \widehat{\overline{x}} \rangle$.

B Connections with predecessors

B.1 Guard (1963) [1]

Guard's system BRA has binary trees, unary and binary functors, an induction schema analogous to Schema F, and propositional connectives $\sim, \supset$ with modus ponens. Guard states Gödel II at p. 16 (Theorem 14 in Guard's numbering). The implication-plus-modus-ponens formulation enables the classical Hilbert–Bernays derivability conditions to be stated internally; Guard's proof follows the DC1-DC3 pattern. Our system omits $\sim, \supset$ and modus ponens entirely, keeping only the equational core.

B.2 Rose (1961) [2]

Rose's equational system is purely equational: no connectives. Proofs are derivations of $t = u$ using computation axioms, congruence, substitution, and an induction principle over unary numerals. Rose states Theorem 18, an equational analogue of Gödel II. His formalism operates on unary numerals $\overline{n}$ rather than binary trees; we adapt his arguments to Guard's tree framework.

Our Theorem 2 corresponds to Rose's Theorem 14 (the object-level reflection of derivability). Our Con_H (Def. 3) is Rose's equational consistency schema translated into the tree setting.

B.3 Hilbert–Bernays derivability conditions

Classical Gödel II uses three derivability conditions:

- (DC1) If $T \vdash \varphi$ then $T \vdash \mathrm{Prov}_T(\ulcorner \varphi \urcorner)$.
- (DC2) $T \vdash \mathrm{Prov}_T(\ulcorner \varphi \to \psi \urcorner) \to \mathrm{Prov}_T(\ulcorner \varphi \urcorner) \to \mathrm{Prov}_T(\ulcorner \psi \urcorner)$.
- (DC3) $T \vdash \mathrm{Prov}_T(\ulcorner \varphi \urcorner) \to \mathrm{Prov}_T(\ulcorner \mathrm{Prov}_T(\ulcorner \varphi \urcorner) \urcorner)$.

In a system with $\to$ and modus ponens, DC2 lets one derive $\mathrm{Prov}_T(\ulcorner \bot \urcorner)$ from $\mathrm{Con}_T \to \bot$ by internalised modus ponens. We have no $\to$ and no modus ponens. Instead, we use Theorem 2 as a *polymorphic DC1-like* reflection, and the diagonal manoeuvre directly within the equational system. In effect we use the sharper Theorem 12 (equational inconsistency of Con alone) where the classical argument uses DC2.

B.4 Joyal's approach [3]

Dijk and Gietelink-Oldenziel (following unpublished ideas of A. Joyal) view Gödel's incompleteness categorically, as a diagonal argument in a cartesian closed category of theories and interpretations. The provability *modality* Prov is realised as a functor; necessitation becomes functoriality, and Gödel I/II become properties of certain diagrams.

The V3 file `Guard/ProvV3.agda` exposes an analogous modal layer on top of thmT: Prov3 H eq records an encoding enc and a correctness witness; the function necessitation wraps a ProofE3 into a Prov3. The Joyal-style reading is compatible with our formalism but not formalised further here.

B.5 V2 of this formalization

The V2 development in `Guard/archive/` formalizes Gödel II with an evaluator thFunTFor that is hypothesis-agnostic. It proves a strictly weaker theorem: its consistency sentence is *refutable by pure computation* because a fabricator mkProofEAny produces encoded proofs of any equation, independent of the actual derivability. V3 closes this loophole via a hypothesis-specific thmT (Section 4), a case26 match on the hypothesis encoding, and a case19V3 middle-term match for transitivity.

C File index

- `Guard/Term.agda`, `Guard/Step.agda`: datatypes of Fun_1, Fun_2, Term, Equation and the Deriv rules; RecP with axioms axRecPLf, axRecPNd.
- `Guard/SubstOp.agda`: substOp with correctness (`substOpEquiv`, `substOpCorrect`).
- `Guard/ThFunTForV3.agda`: the dispatcher, 29 cases; case26, case19V3, case23V3.
- `Guard/Thm14EV3.agda`: Theorem 14 V3.
- `Guard/SubstTForPrecompV3.agda`: diagonal machinery (templateCode, crTC, gs_{V3}, cGS, cGSisCS).
- `Guard/GodelIV3.agda`: `godelIDerivV3`, `diagFTargetV3`.
- `Guard/ProvV3.agda`: Prov3, necessitation, diagContradict.
- `Guard/GodelIIV3.agda`: conSentenceV3, conToBotV3, godelIIPositive, godelIIV3, godelIIConAlone.
- `Guard/GodelIITestV3.agda`: negative acceptance test.

Every file typechecks under `--safe --without-K --exact-split` in under 10 s with the `agda-2.9.0` binary. No postulates.

References

[1] D. Guard, *Simplified foundations for mathematical logic*, notes, 1963.

[2] H. E. Rose, *On the consistency and undecidability of recursive arithmetic*, Z. Math. Logik Grundlagen Math. **7** (1961), 124–135.

[3] D. Dijk and M. Gietelink-Oldenziel, *Gödel's incompleteness after Joyal*, arXiv:2004.10482, 2020.